

DSA

LeetCode Notes

Pattern Recognition, Templates, and Worked Solutions for Common Problem Types

Java Agentic AI — Knowledge Base Reference

1. How to Practice Effectively

- Group problems by pattern, not by random difficulty order — recognizing 'this is a sliding window problem' is 80% of the battle.
- Time-box each problem (e.g. 25-30 minutes); if stuck, read the pattern hint, not the full solution, and try again before looking at code.
- Re-solve problems you've already done a week later without looking — recall under time pressure is what interviews actually test.
- After solving, always write down: which pattern was this, what was the trick/insight, and what edge case did you miss on the first attempt.

2. Arrays & Strings

2.1 Two Sum (Hash Map Pattern)

```
int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();    // value -> index
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[]{seen.get(complement), i};
        }
        seen.put(nums[i], i);
    }
    return new int[]{-1, -1};
}
```

Note: Time: $O(n)$, Space: $O(n)$. Insight: trade space for time by remembering what you've seen instead of a nested $O(n^2)$ check for every pair.

2.2 Maximum Subarray (Kadane's Algorithm)

```
int maxSubArray(int[] nums) {
    int maxSoFar = nums[0], currentMax = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentMax = Math.max(nums[i], currentMax + nums[i]); // extend or restart
        maxSoFar = Math.max(maxSoFar, currentMax);
    }
    return maxSoFar;
}
```

Note: Insight: at each position, decide whether extending the previous subarray helps or whether starting fresh from the current element is better — if *currentMax* goes negative, it can never help a future sum, so restart.

2.3 Product of Array Except Self

```
int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    result[0] = 1;
    for (int i = 1; i < n; i++) result[i] = result[i - 1] * nums[i - 1]; // prefix products
```

```

    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= suffix;           // multiply in suffix product
        suffix *= nums[i];
    }
    return result;
}

```

Note: No division allowed variant: build prefix products left-to-right, then multiply in suffix products right-to-left — $O(n)$ time, $O(1)$ extra space (excluding the output array).

3. Sliding Window

```

// Minimum window substring containing all characters of t
String minWindow(String s, String t) {
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    Map<Character, Integer> window = new HashMap<>();
    int have = 0, needCount = need.size();
    int left = 0, minLen = Integer.MAX_VALUE, minStart = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        window.merge(c, 1, Integer::sum);
        if (need.containsKey(c) && window.get(c).intValue() == need.get(c).intValue())
            have++;

        while (have == needCount) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minStart = left;
            }
            char leftChar = s.charAt(left);
            window.put(leftChar, window.get(leftChar) - 1);
            if (need.containsKey(leftChar) && window.get(leftChar) < need.get(leftChar))
                have--;
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart + minLen);
}

```

Note: Pattern signature: 'find the smallest/largest substring/subarray satisfying X.' Expand right to gather what's needed, then shrink left as much as possible while still valid, recording the best window each time it becomes valid.

4. Linked Lists

```

// Reverse a linked list
ListNode reverseList(ListNode head) {
    ListNode prev = null, current = head;
    while (current != null) {

```

```

        ListNode next = current.next;
        current.next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

// Merge two sorted linked lists
ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(-1);
    ListNode tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
        else { tail.next = l2; l2 = l2.next; }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}

```

Note: The dummy-head trick avoids special-casing the first node — always initialize a placeholder node before the real head and return `dummy.next` at the end.

5. Trees

```

// Validate a Binary Search Tree
boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

boolean validate(TreeNode node, Integer lower, Integer upper) {
    if (node == null) return true;
    if (lower != null && node.val <= lower) return false;
    if (upper != null && node.val >= upper) return false;
    return validate(node.left, lower, node.val) && validate(node.right, node.val, upper);
}

// Lowest Common Ancestor (binary tree, not necessarily BST)
TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) return root; // p and q found in different subtrees
    return (left != null) ? left : right;
}

```

Note: Common trap for BST validation: comparing only against the immediate parent instead of propagating the full valid (`lower`, `upper`) bound range down the recursion — a node deep in the left subtree must still respect an ancestor much higher up, not just its direct parent.

6. Backtracking

```
// Generate all valid combinations of n pairs of parentheses
List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    backtrack(result, new StringBuilder(), 0, 0, n);
    return result;
}
void backtrack(List<String> result, StringBuilder current, int open, int close, int max) {
    if (current.length() == max * 2) {
        result.add(current.toString());
        return;
    }
    if (open < max) {
        current.append('(');
        backtrack(result, current, open + 1, close, max);
        current.deleteCharAt(current.length() - 1);
    }
    if (close < open) {
        current.append(')');
        backtrack(result, current, open, close + 1, max);
        current.deleteCharAt(current.length() - 1);
    }
}
```

7. Graphs

```
// Number of islands (connected components in a grid)
int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                count++;
                sink(grid, r, c); // mark this whole island as visited
            }
        }
    }
    return count;
}
void sink(char[][] grid, int r, int c) {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] != '1')
        return;
    grid[r][c] = '0';
    sink(grid, r + 1, c); sink(grid, r - 1, c);
    sink(grid, r, c + 1); sink(grid, r, c - 1);
}
```

8. Dynamic Programming Classics

```
// Climbing stairs - Fibonacci in disguise
int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
```

```

        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}

// Coin change - minimum coins for a target amount
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);    // 'infinity' sentinel
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}

```

9. Pattern Recognition Cheat Sheet

Signal phrase in the problem	Likely pattern
Sorted array, find a pair/triplet	Two pointers
Longest/shortest substring or subarray with a condition	Sliding window
Detect a cycle in a linked list	Fast & slow pointers
All subsets / permutations / combinations	Backtracking
Top K elements / Kth largest	Heap (PriorityQueue)
Minimum/maximum number of ways, optimal value with choices	Dynamic programming
Range sum queries, frequent updates	Prefix sum / Fenwick tree
Connected components, shortest path in unweighted graph	BFS/DFS
Scheduling / interval overlap	Sort by end time + greedy, or sweep line
Next greater/smaller element	Monotonic stack

10. Common Mistakes in Interviews

- Not clarifying constraints before coding (can input be empty? negative numbers? duplicates?).
- Jumping straight to code without stating the approach and complexity out loud first.
- Off-by-one errors in loop bounds — always trace through a size-1 and size-2 example by hand.
- Forgetting to handle the empty/null input edge case explicitly.
- Not testing the solution against the example before declaring it complete.